An Embedding-Space Perspective Towards a Defense-in-Depth Mechanism for Prompt-to-SQL Attack (Prompt2SQLa)

by

Zi Han Ding

Bachelor of Science in Computer Science and Computational Biology, University of Pittsburgh, 2024

Submitted to the Graduate Faculty of

the School of Computing and Information in partial fulfillment

of the requirements for the degree of

Master of Sciences

University of Pittsburgh

2026

UNIVERSITY OF PITTSBURGH

SCHOOL OF COMPUTING AND INFORMATION

This thesis was presented

by

Zi Han Ding

It was defended on

May 11, 2026

and approved by

Dr. Alexandros Labrinidis (Thesis Advisor), Department of Computer Science

Dr. Panos Chrysanthis, Department of Computer Science

Dr. Lorraine Li, Department of Computer Science

Copyright © by Zi Han Ding
2026

An Embedding-Space Perspective Towards a Defense-in-Depth Mechanism for Prompt-to-SQL Attack (Prompt2SQLa)

Zi Han Ding, M.S.

University of Pittsburgh, 2026

Since the introduction of the transformer architecture, Large Language Models (LLMs) have seen rapid progress in their applications. However, both proprietary and open-source models suffer from a vulnerability to prompt injection, despite the increased attention and investment towards Artificial Intelligence (AI) security. One particular application of LLMs that is vulnerable to prompt injection is the text-to-SQL (Text2SQL) interface. LLMs have dominated multiple Text2SQL benchmarks across multiple domains. The interest in bringing natural language interfaces to database systems is rising, while the security problem remains stagnant.

In this thesis, we address the security vulnerabilities of Text2SQL interfaces through the embedding space. Given that LLMs can generate benign and malicious queries from embeddings, the embedding space must contain information that led to the response. This thesis investigates a finer classification of malicious prompts towards a Defense-in-Depth system.

We introduce a taxonomy and an explainability pipeline in the context of security and Prompt-to-SQL Attacks (Prompt2SQLa). The taxonomy allows us to break down the current binary classification into more granularity for a Defense-in-Depth approach to Prompt2SQLa. We also empirically point out the out-of-distribution (OOD) problem in prior work and provide a concrete direction towards a possible solution for an OOD-robust classification method.

Table of Contents

Preface ix

1.0 Introduction 1

1.1 Contributions 3

1.2 Roadmap 4

2.0 Background 6

2.1 Transformer Architecture 6

2.2 SQL 8

2.3 SQL Injection 8

2.4 Prompt Injection 9

2.5 Embedding Space 10

3.0 System Model 11

4.0 Methodology 13

4.1 Synthetic Label Generation 14

4.2 Few-shot Nearest Centroid Analysis 15

4.3 Classification 15

4.4 Dimension Reduction 17

5.0 Implementation 18

5.1 Tech stack 18

5.2 Dataset 18

5.3 Few-shot Nearest Centroid 21

5.4 Loss function 21

5.5 Additional Hyperparameters 21

5.6 Model Architecture 22

5.7 Metrics 22

6.0 Results 24

6.1 Nearest Centroid 24

6.2 Dimension Reduction 24
6.3 Classification 25
6.3.1 Spot Sample Testing 25
6.3.2 Confusion Matrix 26
6.3.3 ROC-AUC Curve 28
6.3.4 OOD Performance 28
6.4 Ablation Study 29
7.0 Discussion 33
7.1 The Attack Types Exist and Show Distinct Islands in Embedding Space . . 33
7.2 Linear Probe Performance and Ablation 34
7.3 Data Imbalance and Low Spot Sample Accuracy 34
7.4 Defense-in-Depth Approach Needed to Comprehensively Counteract the Modern Landscape of Attacks 35
7.5 Explainability Analysis 36
8.0 Future Work 38
8.1 Matryoshka Representation Learning 38
8.2 Applied Setting 38
8.3 Similar Analysis Pipeline Applies to Other Domains than Security 39
9.0 Conclusions 40
Appendix. Examples and Results 41
A.1 Prompt Injection Sample 41
Bibliography 44

List of Tables

Table 1: Prompt2SQLa Attack Definitions 14
Table 2: Breakdown of Spot Sample Analysis 19
Table 3: Individual Class Distribution of Datasets 20
Table 4: Total Binary Distribution of Datasets 20
Table 5: Spot Sample Test Results 28
Table 6: ROC AUC Scores for Test Set 29
Table 7: Optimal Thresholds for Each Class Based on ROC AUC Scores 30
Table 8: Frozen Encoder Parameter Count and Embedding Output Dimensions . 32

List of Figures

Figure 1: System model architecture overview with numbered steps to guide understanding 11

Figure 2: Model Architecture 22

Figure 3: Few-shot centroid analysis based on embedding output on the six classes - five malicious and one benign class. The dotted line indicates the random-guess baseline of 16.67%. 25

Figure 4: Reducing dimensions from 768 to 50 via PCA, followed by unsupervised UMAP. The data points are colored according to their labels. Performed on the imbalanced dataset. 26

Figure 5: Reducing the dimensions from 768 to 50 via PCA, followed by supervised UMAP. This separated the attack types. Performed on the imbalanced dataset. 27

Figure 6: Trained with a linear probe on 50 epochs for 12 minutes and 30 seconds on LLM labelled data from SQLShield only entries in the training set. Evaluated on the imbalanced dataset. 30

Figure 7: Trained with a linear probe on 50 epochs for 12 minutes and 30 seconds on LLM labelled data from SQLShield only entries in the training set. Evaluated on the imbalanced dataset. The dotted line represents a random baseline of 0.5. All values are far above 0.5. 31

Figure 8: Comparison of linear probe performance across different encoders with SQLShield baselines. 32

Figure 9: The prompt caused the app to run a DELETE statement. 42

Figure 10: The prompt caused the app to run a sleep statement. 42

Figure 11: The prompt caused the app to leak the schema. 43

Preface

I would like to acknowledge and thank the guidance and critiques of Dr. Alexandros Labrinidis in the production of this work. I am incredibly grateful for his continued support throughout my academic career.

I would like to thank Dr. Lorraine Li and Dr. Panos Chrysanthis for serving on my committee and for their insightful feedback during the defense process.

I would like to thank Max Hayes for his work in implementing the sample system model - developed as part of his capstone within the ADMT lab - which is used in this document to illustrate the context and demonstrate the technique.

Finally, I would like to thank my family for their support of my academic career and encouragement.

1.0 Introduction

After the successful launch of ChatGPT 4 in 2023[24], Large Language Models (LLMs) are increasingly integrated into academia, the corporate world, government, and everyday life. Success in several applications, such as text generation and chatbots for interfacing with complex systems and documents, is an ongoing effort. A natural language interface for these systems is analogous to Structured Query Language (SQL) for database systems. Both interfaces attempt to ease the access, modification, and management of databases. While the development of SQL and related tools has increasingly shifted away from use by non-experts, the development of LLMs has shifted towards use by non-experts. Researchers have attempted to combine the natural language interface of LLMs with the more complex language of SQL via Text to SQL (Text2SQL) [39, 18] as a natural extension, since SQL is still similar to English, unlike other programming languages. This is becoming increasingly successful with top contenders reaching the 80th percentile in large cross-domain benchmarks[18]. Allowing Text2SQL interfaces to create queries, execute queries, and interpret the results returned without an expert in the overall pipeline. For the remainder of this thesis, when Text2SQL interfaces are mentioned, we refer to this implementation, where everything is abstracted and automated, without a human expert reviewer.

Despite the success of Text2SQL interfaces or perhaps because of it, Text2SQL creates a significant security problem. LLMs are shown to be vulnerable to prompt injections[33, 26, 25][1], where sensitive data and specified outputs can be manipulated using prompt engineering. SQL has its own SQL Injection attacks, which range from data exfiltration to denial-of-service (DoS) on database systems [11]. Although solutions exist for these problems, due to mismanagement of database systems and inadequate security practices, SQL injection remains a top problem in modern applications[27][2]. A Text2SQL interface, which combines LLMs and SQL, inherits vulnerabilities from both worlds. It is possible to completely deny access by implementing SQL-only solutions that block all forms of attack. While

[1] https://genai.owasp.org/llm-top-10-2023-24/
[2] https://owasp.org/Top10/2025/

this approach works, there is a significant trade-off in utility for any Text2SQL interface when implementing strict guardrails. Other naive, default solutions, like system prompts, can often be bypassed through prompt engineering. Although security remains a significant issue for Text2SQL, with no known all-in-one solutions, the drive towards LLM integration has led to significant data breaches due to rapid deployment at both large and small companies[19, 2, 20][3].

The most recent and state-of-the-art (SOTA) solutions focus on a few directions:

- *Naive LLM-based classification:* In this approach, the LLM gives a second pass on the provided prompt before generating any SQL statements. If the prompt is malicious, the LLM stops generation[31]. While this approach has shown success in constrained, high-fidelity environments, additional studies highlight a significant performance difference when transitioning to an unstructured, real-world setting[4, 8]. In addition, the approach increases the execution time by a non-trivial amount due to increased LLM usage, limiting its applicability.
- *Abstraction of Database Systems:* Through abstracting the database schema, this effectively prevents data leakage via prompt injection, as the LLM does not have access to the schema as part of its system prompt. Any SQL statement generated is then passed through an abstraction layer before being used on the real database[1]. While this attempts an engineering approach to Prompt-to-SQL Attack(Prompt2SQLa), it does so at the cost of utility and does not prevent a malicious actor from attacking. It acts as a roadblock and not a solution.
- *Embedding-based Filtering:* By examining the embedding space of the input prompt, it is possible to determine whether the prompt is malicious or benign, either through semantic similarity[8] or machine learning methods on top of the embedding space[4, 31]. This approach can stop most attacks without incurring significant resource costs and at a very low execution time. We find that these approaches, including our prior work, suffer from out-of-distribution(OOD) errors. For our prior work, without expanding the list, the performance drops significantly. For machine learning based approaches, these fine-tuned models also have significant performance drops; furthermore, you cannot improve

[3]https://msrc.microsoft.com/update-guide/vulnerability/CVE-2025-32711

them without additional fine-tuning. Considering all methods report a 99% F1 score on their own data, this is a significant issue.

This thesis focuses on embedding-space filtering methods. Specifically, current methods only output a binary classification-whether the prompt is malicious or not. When analyzing the attacks, there is no difference between a prompt that attempts to read the schema and one that attempts to perform major modifications across the database. Being able to classify different attacks is key to a Defense-in-Depth (DiD) approach. In addition to solving this issue from the ground up and identifying the core mechanisms that prompt injection, whether Prompt-to-SQL Attack (Prompt2SQLa) or not, there is a need to investigate the explainability of prompt injection. Binary classification methods do not afford the room for explanation.

The natural extension is to explore the embedding space that underpins these approaches. We build on our prior work[8], and successfully explored and investigated a DiD approach that addresses many of the core issues with current methods, as well as provides a comprehensive analysis of the embedding space with multiple methods that arrive at the same conclusion - the necessity to investigate and apply a DiD approach instead.

1.1 Contributions

This thesis analyzes the embedding space in relation to Prompt2SQLa attacks from a DiD perspective and lays the groundwork for further explainability analysis. The main contributions include:

- We created a taxonomy of Prompt2SQLa attack types for additional analysis.
- We performed a comprehensive analysis of the embedding space using the taxonomy and multiple approaches to confirm the feasibility of a DiD approach as well as the taxonomy's own validity.
- We trained a linear probe that demonstrates the potential of a DiD approach.

- We performed ablation studies on multiple encoders to confirm the consistency of representations across different models.
- We compare the downstream performance of the linear probes against existing defenses and highlight the out-of-distribution issue with fine-tuned models.
- We highlight the out-of-distribution robustness with a DiD approach.

1.2 Roadmap

The remainder of this thesis is organized as follows:

- *Chapter 2* - provides a comprehensive review of relevant literature. We include seminal papers outlining the structure and evolution of applications of the transformer architecture in text and code generation. We establish the foundations and applications of SQL and database systems before assessing the evolution of SQL injection attacks and mitigation strategies. We then explain the recent prompt injection attacks and mitigation strategies. Finally, we move to the embedding space and present relevant research for assessing the encoder output.
- *Chapter 3* - a detailed overview of the specific system model that we use for our examples. We explain how we envision a modern Text2SQL application would function and provide breakdowns of how a prompt is processed.
- *Chapter 4* - a high-level breakdown of different methods and their objectives in the effort of showing the potential of embedding space in a DiD and an explainability perspective. We explain how the methods proposed will be different from previous works in this niche, as well as show a comprehensive coverage of current embedding-related assessment and metrics.
- *Chapter 5* - a detailed breakdown of specific tools, frameworks, and hardware used in the experiments. We justify and discuss the experiment setup. In addition, a breakdown of the datasets used and class distribution, as well as any synthetic data generation methods, is presented.

- *Chapter 6* - Presentation of experimental results, highlighting findings and trends. We provide visualizations and analysis of different experiments. In addition, we present our ablation study results to support existing figures, and finally, we validate the hypothesis of distinct concepts in embedding space.
- *Chapter 7* - A detailed discussion of key findings and their impact. We extend the analysis beyond a security perspective and provide additional insights into failed experiments and the thought process behind initial experiment designs. We also discuss the insight behind having distinct concepts in embedding space and how it influences a DiD strategy.
- *Chapter 8* - Any potential extension of the research that would provide additional explainability insights or practical applications.
- *Chapter 9* - A summary of the entire thesis. We provide a high-level overview of each individual chapter and key findings.

2.0 Background

To understand the problem of Prompt2SQL in its entirety, including the cause and current solutions, this section provides the theoretical foundations for the rest of the paper. Sections 2.1 and 2.2 will cover the two tools used in modern computer science: Transformers and SQL. Sections 2.3 and 2.4 dive into what is being exploited and how it is being exploited. Section 2.5 will cover the concepts needed to understand the origins of the proposed solutions and why existing solutions fail to achieve good results outside an experimental setup.

2.1 Transformer Architecture

Since its inception in 2017[34], the transformer architecture has taken over the machine learning landscape almost entirely. Largely due to the attention mechanism, which broke several bottlenecks and redefined how we process sequential data[3]. Before the introduction of attention, we relied on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks[21, 16]. These original network architectures suffer from a few bottlenecks due to how they process and understand data inherently:

- *Vanishing Gradient* - This is a problem solved largely by ResNet. The older network architectures shared weights, which, when too small, cause the gradient to vanish and, when too large, cause the gradient to explode. LSTM introduced an additional forget gate to provide regularization, but it did not switch from Sigmoid and Tanh activations.
- *Long-Range Dependency* - This is an inherent language vs architecture situation. The subject of a sentence is at the beginning, but the verb and the object are at the end. The initial subject loses the mathematical signal as the sentence gets longer.
- *Information Bottleneck* - Older networks relied on passing a single vector between layers. For longer sequences, this means compressing information into a limited amount of representation. The pigeonhole principle makes it clear that if a vector can only represent n different words in their entirety, the $n + 1$ word will cause information loss.

- *Efficiency Bottleneck* - These older network architectures are also sequential. At step 10, it must have the hidden state from step 9. So it cannot utilize modern hardware to its fullest. There is no parallelism to exploit.

Each of these problems was addressed in several related research papers prior to the transformer architecture, synthesizing the various breakthroughs. The vanishing gradient problem was largely addressed by AlexNet and ResNet[17, 13]. AlexNet popularized the ReLU activation, which replaced saturating activation functions such as sigmoid or tanh. AlexNet also showed that deep neural networks are viable. ResNet introduced residual connections that, by architecture, addressed the vanishing-gradient problem. These residual connections ensure proper backpropagation, regardless of the weights' magnitude. Turning the multiplication of gradients into a summation.

The attention mechanism[3] addressed the problems of long-range dependencies and information bottlenecks. By computing an attention score, even if the subject is at the beginning of the sentence, its hidden state remains relevant to the decoder, rather than being lost to compression. It dynamically assigned the attention scores as the sequence grew.

This leads to a clear scaling bottleneck with RNNs. The transformer architecture addresses this through its attention engine. Instead of using a sequential evaluation, it computes the entire attention matrix in a single step at the beginning. Residual connections facilitate gradient flow through very deep encoder and decoder architectures. Taking full advantage of modern hardware.

Of particular interest to this work, the improvement to the transformer architecture significantly affects the embedding space. It introduces emergent behaviors outside of just text[10], and records more information than just the words. The hidden domain knowledge and intentions are translated to the embedding space, which supports more utility than just passing the information to a decoder.

2.2 SQL

The structured query language (SQL) is a relational model to interact with database management systems (DBMS). Originally, database systems required either hierarchical or network-based navigation. This limited the capabilities and inherent relationships between data. A relational model for interacting with database systems was proposed to account for these complex relationships[6]. This was then developed into a full relational database system called System R. Structured English Query Language (SEQUEL) was subsequently created to interact with System R[5]. This was the predecessor to modern SQL. A language designed with interpretability in mind, as suggested by the name. Over time, to prevent fragmentation of SQL dialects, standards were introduced that led to the SQL landscape of today.

Despite its original intent as a highly accessible interface, modern SQL has evolved into a language accessible only to experts, despite its English-like syntax[23]. This led to the current push for a complete natural-language-based interface, tested through benchmarks comprising large, cross-domain questions and databases[39, 18].

However, this does not mean that modern SQL is now less interpretable than programming languages. It is still in a very recognizable syntax to someone who understands English. Modern SQL is something between a natural language and code, even if its relative position has shifted towards code. This makes it appropriate for a natural language processor like an encoder or decoder to understand and output.

2.3 SQL Injection

As SQL became more mainstream, dynamic programming of SQL inputs became the norm. Instead of writing directly to SQL, input fields are treated as variables in SQL statements to enable automated database operations. This is used extensively in security-related purposes such as authentication, authorization, and accounting. However, dynamically populated SQL fields can be manipulated to execute queries and extract data. This discovery of

a new attack surface has led malicious actors to exploit SQL to launch a variety of attacks. Using SQL specifically at every stage of the attack due to how integrated database systems are in modern computing.

This is known as SQL injection. Extensive work by both public and private institutions and organizations has documented procedures for detecting and mitigating these attacks. Such as input validation and sanitization, parameterized queries, and testing frameworks that collectively prevent malicious actors from abusing SQL injection vulnerabilities. However, although this is largely a solved problem, with solutions often included by default in modern SQL and related frameworks, the rollout of these solutions is slow. Even in 2025, it remains among the top 10 issues in modern websites[27].

2.4 Prompt Injection

Prompts, composed of entirely natural language, are also capable of exploiting systems for a variety of purposes, much like SQL injection. LLMs do not support security inherently. In fact, their architecture inherently supports insecurity. The better they understand the input tokens, the more vulnerable they are to prompt injection. Whether through coercion, urgency, or natural-language structure, LLMs and their downstream tasks are vulnerable to prompt injection.

Prompt injections are also inherently difficult to defend. Unlike SQL injection attacks, there is no input validation or sanitization. No parameterized queries, as LLMs process the prompt in its entirety. The only possibility is through testing. But this is exponentially more expensive and slower than testing SQL queries. Furthermore, unlike SQL queries, which have a specific goal, natural language queries are open-ended. There is no specific target, like in SQL queries, to build a testing framework around. Prompt injection has become a top problem across many applications and frameworks.

2.5 Embedding Space

The output of an encoder results in a high-dimensional tensor representation of the data. This is used later by the decoder to produce an output. Forming the two components that make up the transformer architecture underlying every LLM.

Originally a static system, it has since changed with the transformer architecture [7]. This later evolved into a full sentence-embedding system using a Siamese network with contrastive learning. Currently, the encoder space is a central focus of explainability research. Research in both representation learning and mechanistic interpretability is focused on encoders and the embedding space[9].

In the context of prompt injection, the embedding space forms the basis of many recent machine-learning-based solutions. In our previous work[8], we classified the prompts in a zero-shot manner by comparing prompts to a fixed list of generic prompts. We calculated a cosine similarity score and used a universal threshold for binary classification. There were flaws with this approach, as outlined in the future works section. We intend to address some of the ideas in the future works section in this thesis, in particular, the individual thresholds for different attacks.

Outside of our previous work, there are a few other notable works that build on an encoder[4, 31]. These works mainly focus on providing a fine-tuned encoder with additional layers for classification. A recurring problem of concern is the consistent, high performance in all three, including our prior work. In all cases, every study reports a near-perfect score (99%) across multiple categories. This is especially suspicious when it comes to fine-tuned models and a zero-shot model. We suspect that these models suffer from serious OOD errors, including our prior work. This has not been extensively studied[35], but it remains a general concern tied to fine-tuning.

3.0 System Model

For this thesis, we assume a simple system model as outlined in figure 1. We break down the system model in its own chapter in order to provide a complete picture of the environment we assume Prompt2SQLa would happen in and how the components of the application can lead to this attack surface.

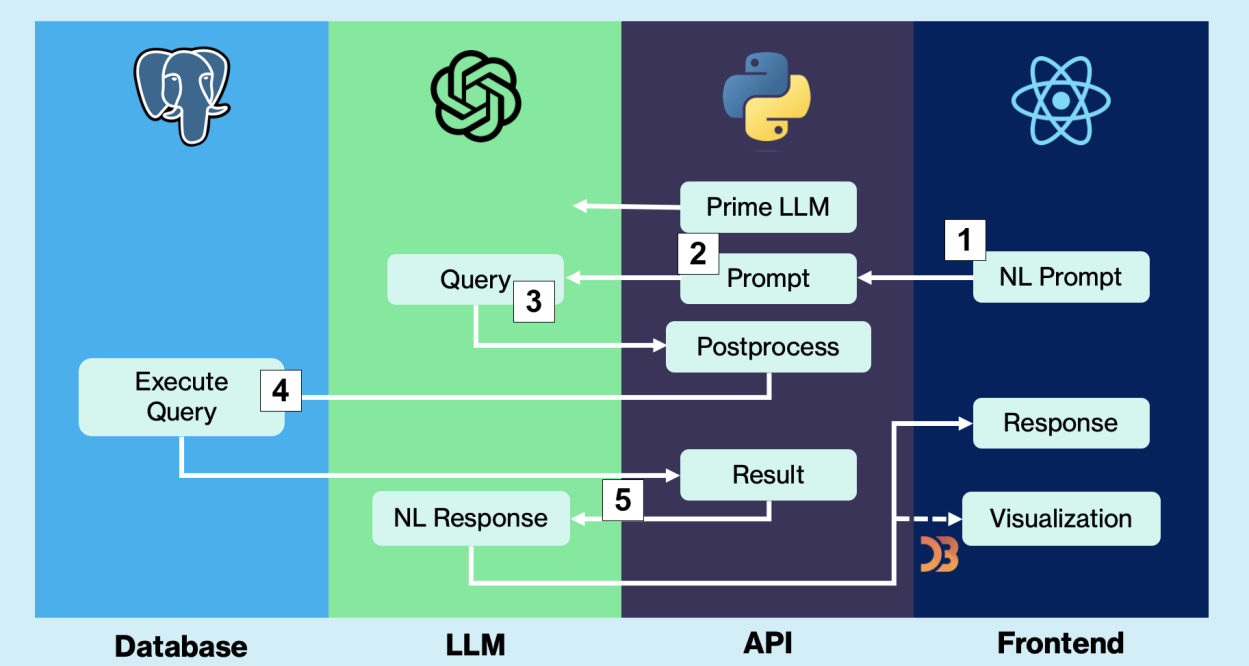

Figure 1: System model architecture overview with numbered steps to guide understanding

In Step 1, the user provides a natural language prompt to the system, for example:

When was the warmest day of September 2023?

The user prompt is then combined with system prompt instructions to form a cohesive Text2SQL attempt in step 2. The system prompt in this case simply instructs the LLM to return a reasonable query that can answer the user prompt in step 3.

An example system prompt would be:

You are given the following schema: ..., provide your answer by generating a SQL query, and if you cannot generate the SQL query, provide a text response.

The LLM would then return the query of

```
SELECT date, tmax FROM pgh_weatherdata
WHERE date BETWEEN '2023-09-01' AND '2023-09-30'
AND tmax IS NOT NULL AND tmax::integer = (SELECT MAX(tmax::integer)
FROM pgh_weatherdata WHERE date BETWEEN '2023-09-01' AND '2023-09-30'
AND tmax IS NOT NULL) LIMIT 1;
```

This query is then executed by the database management system (DBMS) software in step 4. We then take the output from the DBMS, send it as another prompt to the LLM, which will format the natural language response that is passed to the user in step 5. This may potentially include additional processing, such as generating figures.

The result from the example query is

```
('2023-09-04', 91)
```

In this case, we would pass the response from the database with an example system prompt of:

Answer with a JSON object. Provide the summary of the results in a concise manner. If the provided answer can be visualized, add an additional field of "vis" with the value true.

Which would give us the natural language response after processing:

The warmest day of September 2023 was September 4th, with a high temperature of 91°F.

We believe this establishes a simple, but important, basic premise of how a typical, automated Text2SQL pipeline would be structured. Although it can be argued that an agentic process can replace steps 2 and 3 of the pipeline, we believe that this version is clearer in establishing context.

4.0 Methodology

To empirically show that these concepts exist in the embedding space individually, rather than as a single holistic group of malicious prompts or a Prompt2SQLa cluster, multiple approaches should be taken, and all should reach the same conclusion: that a finer subset of representations exists rather than a general label. While different approaches build on one another, they should also be evaluated in isolation.

To explore this space, we take several frontier techniques for evaluating an embedding space. Namely:

- Zero-shot classification
- Few-shot Nearest Centroid
- Linear Probes
- Clustering

Zero-shot classification is the most direct method of evaluating the strength of an embedding output. To evaluate whether concepts separate into distinct spaces, cosine similarity with a threshold can be used to assess their differences. In our previous study, we applied this technique against multiple types of Prompt2SQLa attacks and achieved significant results. While it was not a direct many-to-one mapping of specific concepts, it was a many-to-one mapping of specific purposes. Such as reconnaissance-related tasks, like "What is the schema?" The performance of a zero-shot classifier at a 0.99 F1 score demonstrates that these individual attacks are closely related in the embedding space[8]. However, this does not directly support the evidence of overarching attack types, only sentence-to-sentence similarity.

To that effort, we constructed a taxonomy of general attack types based on the MITRE attack framework[22][1] and previous taxonomies of SQL injections for this study in table 1[11].

If concepts in the embedding space are related to defense, they should fall under one of

[1]https://attack.mitre.org/

these broader attack categories. A separation of attack types not only helps with synthetic labeling techniques but also serves as an initial step for a DiD approach to prompt injection.

Table 1: Prompt2SQLa Attack Definitions

Attack Type	Definition
Reconnaissance	Attacks that seek to obtain metadata about the database system, verify the existence of identifiers/rows, and identify current user privileges. Actions that map the environment.
Modification	Attacks that seek to compromise data integrity. Inserting, deleting, or modifying existing data.
Exfiltration	Attacks that seek to extract information from the database to an unauthorized location.
Escalation	Attacks that seek to elevate user privileges.
Disruptions	Attacks that target the availability of database systems.

While these labels are comprehensive, an equal distribution of these attack types is not expected. Given the SQL injection context, for example, privilege escalation attacks are inherently rarer than reconnaissance attacks, as reconnaissance attacks are far more accessible.

4.1 Synthetic Label Generation

To facilitate the analysis approaches in this paper, we need a multi-labeled dataset that contains the classes outlined above. However, none of the existing datasets contain attack type classes, and manually labeling each and every row is not scalable. Instead, we propose an LLM-based method to synthetically label attack types. This comes with its own caveats, such as bias. By leaving labeling to a generic LLM, we are susceptible to training data patterns. In order to mitigate bias and verify the effectiveness of LLM-based labeling, we performed a spot sample analysis to ensure that labels are accurate.

Instead of limiting each prompt to a single class of attack, we chose to allow multiple classes for a single attack. This is because real-life attacks are complex in purpose. Retrieval of metadata can be for reconnaissance purposes or the whole point of an attack. We cannot rule out this possibility, so we are allowing multi-class labels.

Naturally, with LLM-labeling, we will not attempt continuous labels, due to architectural constraints and issues with transformers that work against numeric values. These issues are multifaceted, including but not limited to: tokenization of numbers, which decouples the concept of magnitude from numeric tokens, an objective function that prioritizes correct vocabulary over correct value, output tokens are generated one at a time instead of a holistic value, etc. If continuous labels are needed, either converting these discrete labels to continuous or another technique, like cross-encoders that assign a similarity score, can be used instead.

4.2 Few-shot Nearest Centroid Analysis

This approach evaluates the effectiveness of two things: Taxonomy Validity and Embedding Space Quality. If the taxonomy is accurate and these attacks can be organized into these distinct categories, then few-shot nearest centroids should show a significant improvement over the random baseline (six classes, so 16.67%). If the embedding space quality is good, then with more samples, the accuracy should increase significantly, as similar concepts should be close to each other in the embedding space.

4.3 Classification

To demonstrate that distinct concepts exist in the embedding space, a linear probe is required. While rule-based classification is helpful, a linear probe on top of a frozen encoder is the most direct way to observe the distinction between the concepts. If a single linear layer can separate the embedding space into distinct regions, then the underlying data pattern

supports more granularity.

Furthermore, one of the goals of this study is to determine whether a DiD detector can be developed for Prompt2SQLa attacks, so using a linear probe as a baseline helps assess how effective a deeper network can be.

Given the multi-label approach and class imbalance, a simple accuracy metric does not provide a comprehensive assessment. This would work well for a binary assessment or for datasets with class imbalance.

In the linear probe approach for multi-label classification in this paper, we chose a logit output for each class. As there are attacks that carry multiple aspects, such as reconnaissance and exfiltration. In general, metadata is treated as reconnaissance, whereas obtaining metadata constitutes an exfiltration attack. Most attacks in the dataset are single-labeled, but given this is possible, we have chosen a classifier that allows multiple labels to be true. With a logit-based output, accuracy and F1 score make less sense. As individual thresholds for each label may differ. This aligns with the DiD approach, and the AUC-ROC score is most suitable for assessing capabilities regardless of the threshold chosen.

However, in order to compare with current models that perform binary classification, we convert the multi-label classifier into a binary classifier by classifying any positive attack labels as malicious and all negative labels as benign. To ensure fairness, we used the same training dataset as SQLShield and evaluated on the same test set. This was done by separating entries in our synthetically labeled training set that were part of the SQLShield dataset. By using SQLShield as a baseline, we will be able to highlight both the OOD problem with SQLShield and OOD robustness with a concept-based approach. We will not address OOD robustness directly in this work, as the focus is on an exploration analysis of the space. A linear probe with five neurons is not meant to out-compete a fine-tuned model or a complex classifier.

We must also ensure the attack taxonomy is not restricted to one encoder, as it could be the specific encoder being exceptional for our purpose. If the attack taxonomy is valid, then a consistent trend should be observed across different encoders for the embedding space. We should also observe a decent performance regardless of embedding size. We do not expect a smaller encoder to have the same performance as the larger encoder, as a larger space

represents more information than a smaller space naturally. We do expect that smaller encoders still maintain signs of OOD robustness.

4.4 Dimension Reduction

Dimensionality reduction is primarily used for visualizations. It is not possible to produce clean clusters in 768 dimensions with BGE-base, or even higher dimensions with other proprietary models. The curse of dimensionality implies that any clustering technique is confounded by distances across different dimensions.

Thus, we applied both supervised and unsupervised approaches using UMAP and PCA. PCA will reduce the 768-dimensional space to a more manageable 50-dimensional space. Then UMAP is used to visualize the clusters.

5.0 Implementation

For all experiments in this thesis, the setups are as consumer-friendly as possible. However, for parallelization for some of the longer tasks, such as few-shot nearest centroids and linear probes, a Google Colabotary standard T4 GPU machine is used. For the frozen encoder, we will use BGE-Base-En-1.5 as it is small enough to run on consumer grade hardware[38]. For our ablation study, we chose E5-base-v2 and all-MiniLM-L12-v2[36, 32]. E5-base-v2 has a similar number of parameters and a different architecture, so they act as evidence that our taxonomy is encoder-agnostic. MiniLM has significantly fewer parameters and half the embedding dimensions. This is evidence for whether our taxonomy is consistent across scales.

5.1 Tech stack

The main library used is PyTorch for machine learning[29]. This is the modern comprehensive library for creating neural networks.

Scikit-learn was used to calculate metrics such as ROC AUC scores[30].

Numpy was used to represent and process tensor data[12].

For visualizations, a mix of livelossplot for monitoring during training and matplotlib and seaborn for evaluation figures is used[14, 37].

Pandas is used for data exploration and analysis[28].

5.2 Dataset

For the dataset, this paper uses synthetic datasets from previous work on DaMiT-SQL and the SQLShield benchmark dataset[8, 4]. Both datasets contain only binary classification labels. For a problem that explores individual concepts in the embedding space, these

datasets will not suffice on their own. In addition, a further round of data cleaning to remove duplicates was performed on both datasets.

We applied an automated labeling approach using Google's Gemini 2.5 Flash Lite model and operated on our entire dataset. We allowed attacks to be labeled as multiple classes since attacks in real-life can also be multifaceted. To verify potential biases in our approach, we spot-sampled 375 random entries and achieved 79.47% accuracy when compared to our manual labels. We acknowledge that this is a low accuracy for labels, thus we implement an additional test to verify the validity of this technique on the spot samples alone. For entries that belong to multiple classes, we considered it correct as long as one of them matched our manual label. In table 2, we observe a class imbalance, which recurs in our later assessments of the datasets as well. However, addressing class imbalance in benchmarks is beyond the scope of this paper.

Table 2: Breakdown of Spot Sample Analysis

Class	Accuracy(%)	Sample Count
Reconnaissance	67.06	85
Modification	94.12	85
Exfiltration	18.18	11
Escalation	-	0
Disruption	100	2
Benign	81.77	192
Total	79.47	375

We then combined our balanced dataset with the SQLShield dataset (combining the train, validation, and test sets into a single dataset), resulting in a multi-labeled dataset. We split it into training, test, and validation datasets, following a 70-15-15 split. The imbalanced dataset was not included because it has a benign-to-malicious prompt ratio closer to that observed in real-world settings.

We have outlined the total number of instances for each class in each split in Table 3. Note that this table will have repeated counts due to multi-labeled entries. The total

malicious vs benign count is shown in table 4. These are not the original distribution, but the distribution after LLM labeling.

While the test set is used for evaluation, the imbalanced dataset is used to generate visualizations and ROC-AUC scores. The test set does not have enough exfiltration and escalation entries to see meaningful trends. We report the performance on the test set, but the performance on the imbalanced set is more representative of general model performance. The SQLShield dataset is used in classification tasks to demonstrate OOD robustness.

Table 3: Individual Class Distribution of Datasets

	Recon.	Modification	Exfiltration	Escalation	Disruption	Benign
Train	1936	2351	91	105	978	6140
Validation	383	498	22	15	216	1351
Test	420	464	21	23	208	1349
Imbalanced	1599	1008	335	61	375	10680
SQLShield (Within Train)	1435	1977	91	102	818	4387

Table 4: Total Binary Distribution of Datasets

	Malicious	Benign	Total
Train	4499	6140	10639
Validation	929	1351	2280
Test	931	1349	2280
Imbalanced	3110	10680	13790
SQLShield (Within Train)	3469	4387	7856

5.3 Few-shot Nearest Centroid

For few-shot nearest centroid analysis, we used the encoder directly. We attempted 1-, 2-, 4-, and 8-shot samples from the training set, running each shot for 500 random seeds. We calculated the mean of each shot and used that as the centroid. The centroids are used to classify additional samples by calculating the distance of the sample to each centroid, taking the centroid with the minimum distance as the predicted class. This was evaluated on the validation set.

5.4 Loss function

For classification, BCELogitLoss is used in place of both the activation function and the loss function. Since this is a multi-label classification problem, we cannot use a softmax function. A sigmoid activation function and BCELoss can cause a Not a Number (NaN) error due to finite precision. Thus, a Log-Sum-Exp trick is used with BCELogitLoss to operate directly on the logits. The formula for BCELogitLoss is as follows:

$$\ell(x, y) = L = \{l_1, \ldots, l_N\}^\top, \quad l_n = -w_n \left[y_n \cdot \log \sigma(x_n) + (1 - y_n) \cdot \log(1 - \sigma(x_n)) \right]$$

- x_n represents the raw scores predicted by the model
- y_n represents the ground truth
- $\sigma(x)$ represents the sigmoid activation function

5.5 Additional Hyperparameters

The Adam optimizer (Adaptive Momentum Estimate) is used with the default learning rate of 1×10^{-5}. This is considered a universal standard for machine learning tasks. We acknowledge that the starting learning rate is lower than necessary for a 5-neuron model;

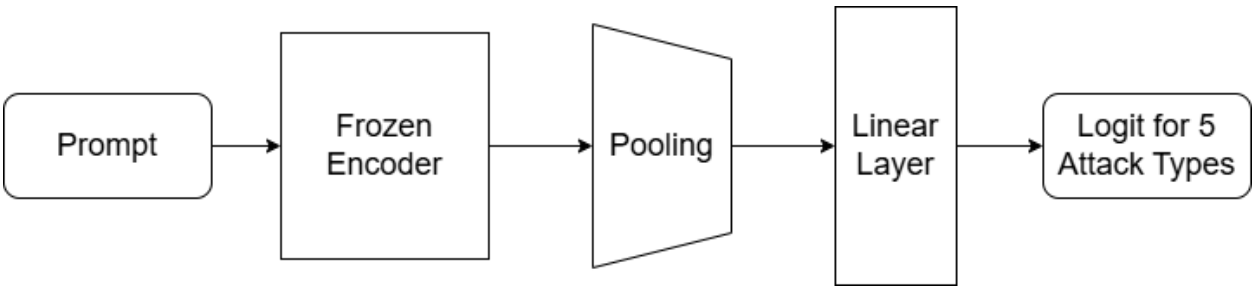

Figure 2: Model Architecture

however, we are running the linear probe on top of a frozen encoder, which requires a more conservative approach.

Following the same logic, we chose to run 50 epochs, a standard number of training cycles.

5.6 Model Architecture

A frozen encoder base is used. These weights are frozen because the study aims to demonstrate the inherent embedding-space concepts and to show that they can be derived and used in DiD defense approaches.

For the linear probe, a pooling layer is applied to the encoder output tensor to produce a vector representation. Then a linear layer with 768 features in and 5 features out is applied. We chose to classify prompts with false on all attack types as benign, so with 5 output features, there are six distinct classes. We illustrate this in figure 2.

5.7 Metrics

There are a few standard metrics we use - Accuracy, F1 Score (which includes both precision and recall), and ROC-AUC score.

We believe that while the rest of the metrics are easy to understand, why they are used, and how they are calculated, there is a special case for the ROC AUC score. We have a benign class that is not predicted by any individual output, but instead by the collective output.

We begin with the ground truth label, which is easily derived; if the sum of the labels is 0, then it is benign. We calculate the benign class probability by subtracting the argmax of all the attack classes. This allows us to run ROC AUC. In reality, we do not rely on the benign threshold at all.

6.0 Results

Based on the approaches outlined in the implementation chapter, several methods reveal key data patterns that favor distinct concepts in the embedding space.

Most notably, across all methods, a positive trend or strong signals are evident.

6.1 Nearest Centroid

For a preliminary analysis of the embedding space, we used a few-shot nearest-centroid approach. Shown in figure 3, this approach showed a consistent and significant upward trend as the number of samples per shot grew. Even with 1-shot centroids, obtained by taking six random samples and using them as centroids, it remains above the baseline of 16.67%. At the highest sample count, it is well above and demonstrates the presence of these concepts in the embedding space.

6.2 Dimension Reduction

A combination of PCA and UMAP worked well for the 768-dimensional embedding space. As shown in both figure 4 and figure 5, distinct clusters are shown. In figure 4, a mixture of concepts is shown on the right, and there is a clear distinction between benign and malicious clusters that can be used for binary classification. Although some benign data points remain within the malicious concept clusters, it is clear that a simple linear layer should be able to separate the binary concepts of malicious and benign.

In addition, applying a supervised UMAP rather than an unsupervised one enables visualization of distinct concept clusters. In figure 5, we observe that reconnaissance and modification form distinct islands. The other attack types, although present, are scattered throughout the space. With more examples, we do see exfiltration forming its own island.

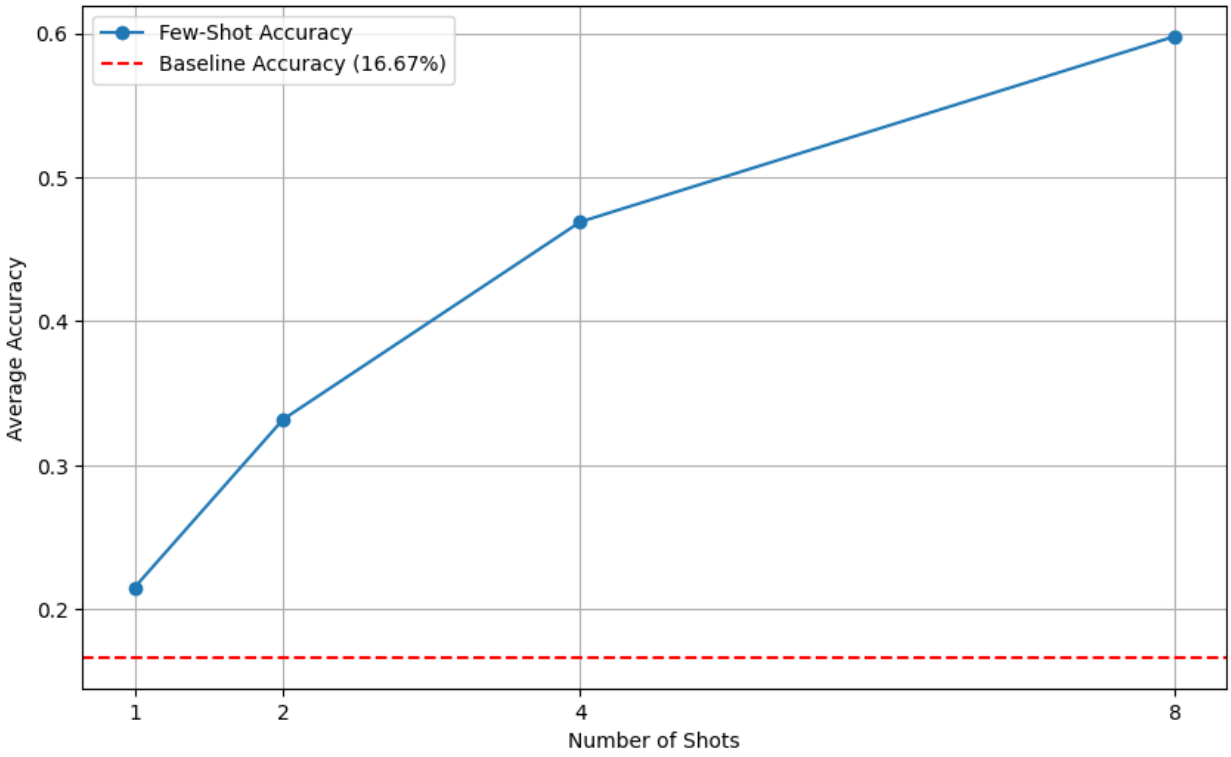

Figure 3: Few-shot centroid analysis based on embedding output on the six classes - five malicious and one benign class. The dotted line indicates the random-guess baseline of 16.67%.

6.3 Classification

6.3.1 Spot Sample Testing

To address concerns about low labeling accuracy, we conducted several tests on the manually labeled spot sample. We present the results in table 5. Both scores are higher than the spot sample baseline.

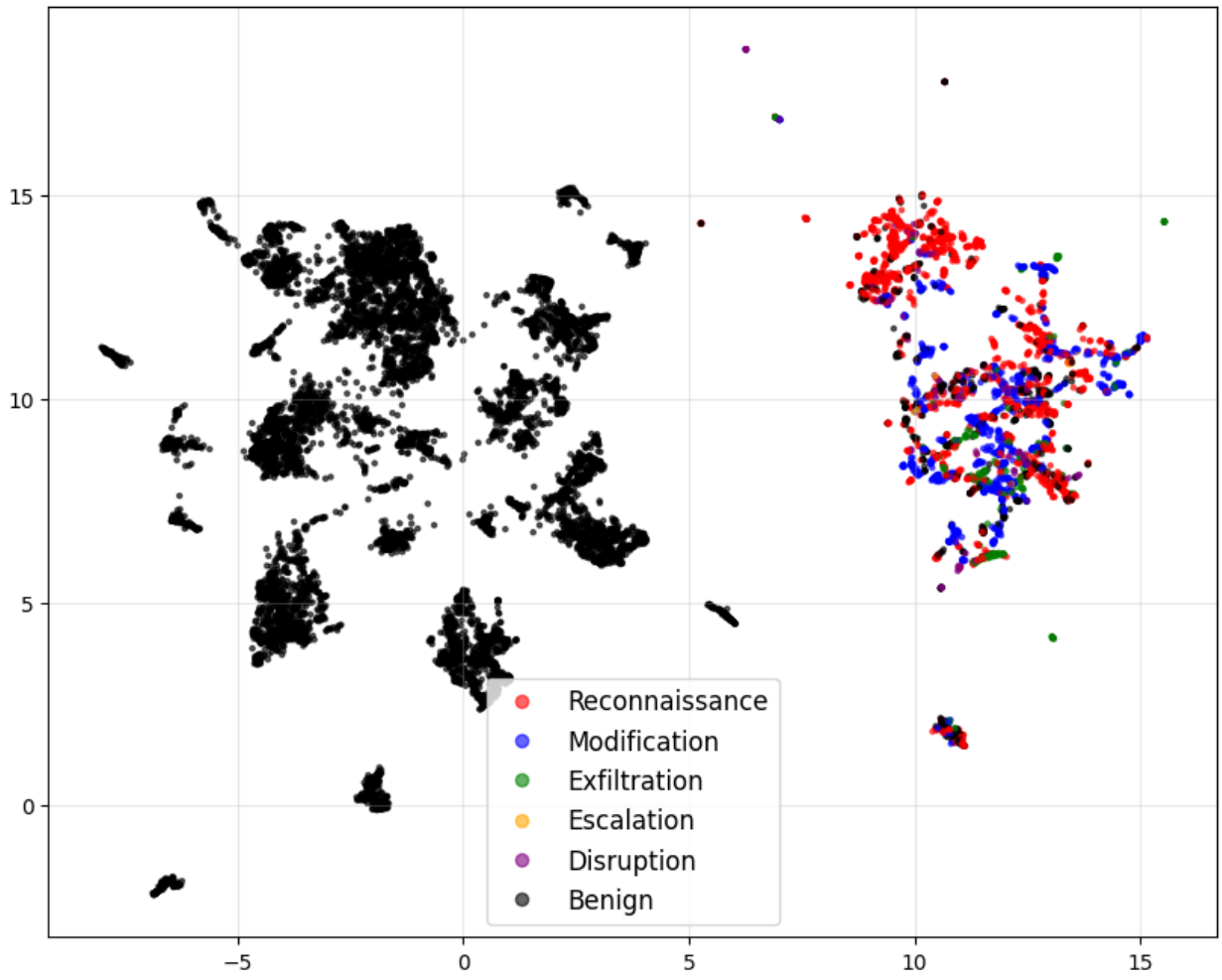

Figure 4: Reducing dimensions from 768 to 50 via PCA, followed by unsupervised UMAP. The data points are colored according to their labels. Performed on the imbalanced dataset.

6.3.2 Confusion Matrix

Table 6 shows the performance on the test set, trained with the train set. With high ROC-AUC scores across the board. Due to the lack of other multi-class classification models for attacks, we are not able to present a baseline. The other metrics are based on the model trained on the SQLShield set to demonstrate OOD capacity, as we believe the test set results support the development of a concept-based model already.

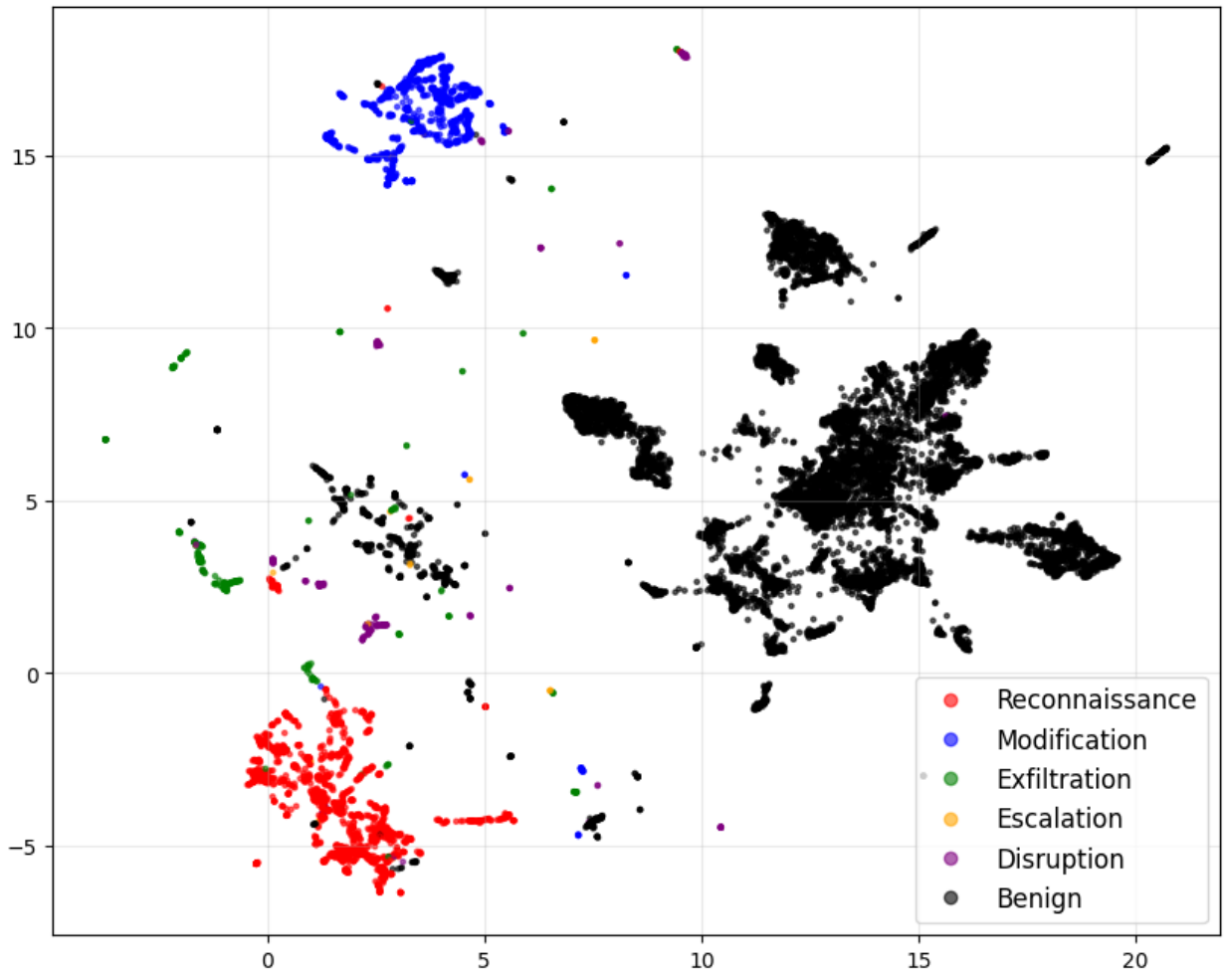

Figure 5: Reducing the dimensions from 768 to 50 via PCA, followed by supervised UMAP. This separated the attack types. Performed on the imbalanced dataset.

Based on the optimal threshold derived from the SQLShield set, shown in table 7. We classified the imbalanced dataset and found that the logit labels do not show a clear separation of the middle three classes of exfiltration, escalation, and disruption in figure 6. These labels were largely classified as other attack types, notably reconnaissance and modification. We generate the confusion matrix by taking the max logit output of all the classes and using it as the label. If none of the logit outputs meet the threshold, it is benign. There are misclassifications, but the benign column shows high recall due to very few false negatives

Table 5: Spot Sample Test Results

Test Type	Percentage Score
Classification Accuracy	89.33
Macro ROC-AUC Average	94.33

(negative being benign; positive being anything labeled as an attack). Notably, exfiltration, escalation, and disruption are misclassified as reconnaissance and modification because the approach used to generate the matrix relies on the highest logit value rather than any matches to present a cohesive confusion matrix that matches our dataset numbers.

6.3.3 ROC-AUC Curve

However, the confusion matrix is not the only metric to consider. We use the same SQLShield dataset and evaluate on the imbalanced dataset for this metric as well. We look at the ROC AUC curve in figure 7. Notably, each class exhibits a clear elbow. This is due to the training data being binary multi-class labels. A regression-based labeling technique is needed to generate smooth curves. There are no curves, or in this case lines, that match the 50% baseline. All classes have significant AUC scores, with a macro-average of 0.8266. The trend cutting off at a certain point is not a sign of improper evaluation but rather the outcome of training on binary multi-class labels.

6.3.4 OOD Performance

Finally, in the downstream binary analysis of model performance shown in figure 8, the linear probe is comfortably above the query-specific shield and only slightly below the prompt shield. Both are in the 90th percentile of F1 scores. The linear probe is significantly higher than SQL Query Shield, indicating OOD issues. The highest being SQL Prompt Shield, but only by less than 3 percent.

Table 6: ROC AUC Scores for Test Set

Attack Class	ROC AUC Score
Reconnaissance	0.9582
Modification	0.9946
Exfiltration	0.9382
Escalation	0.8751
Disruption	0.9837
Benign	0.9794

6.4 Ablation Study

We also observe in figure 8 that encoders of a similar size maintain a similar level of performance and that a smaller encoder, while suffering performance losses, still shows signs of OOD resistance.

We list the parameter counts and embedding dimensions in table 8 for comparison. Given the limited number of encoders compared, we are not producing a visualization.

Table 7: Optimal Thresholds for Each Class Based on ROC AUC Scores

Attack Class	Optimal Threshold
Reconnaissance	0.2186
Modification	0.3098
Exfiltration	0.0175
Escalation	0.0202
Disruption	0.1962
Benign	0.6905

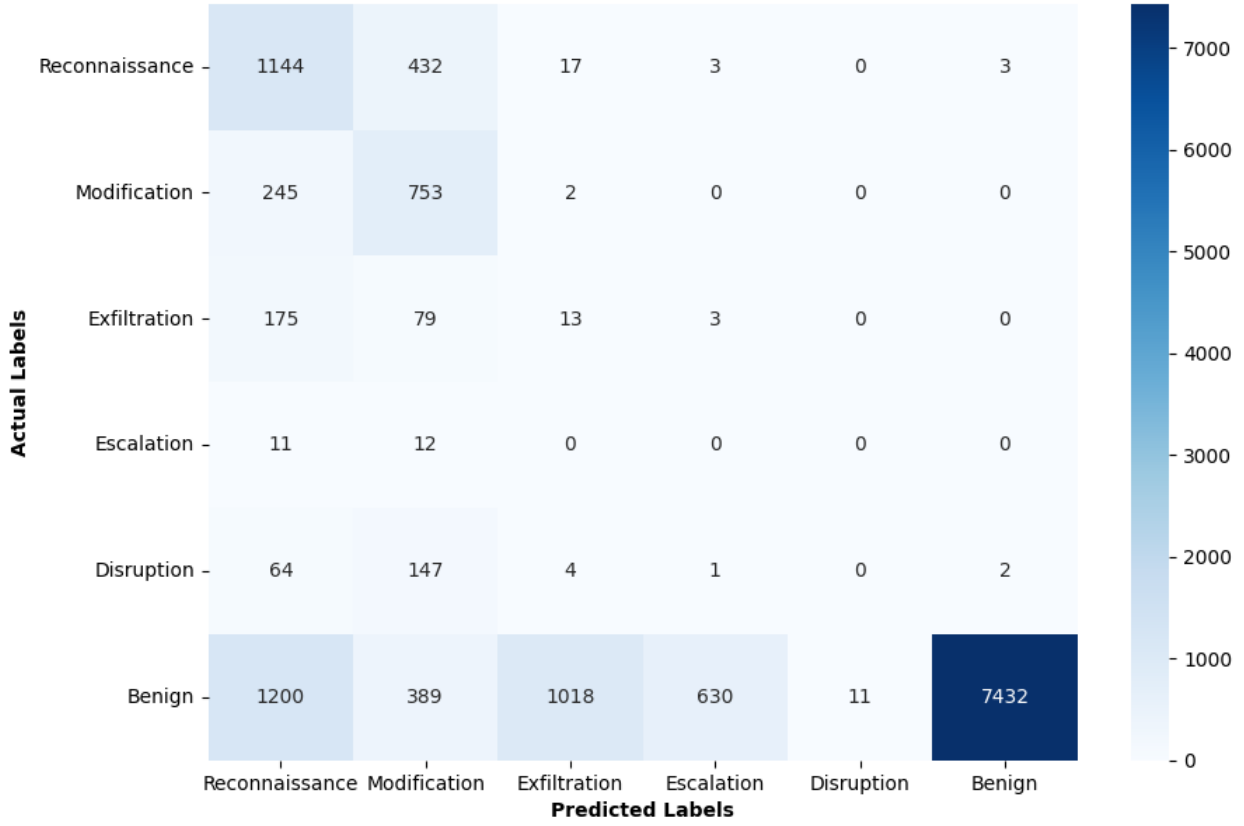

Figure 6: Trained with a linear probe on 50 epochs for 12 minutes and 30 seconds on LLM labelled data from SQLShield only entries in the training set. Evaluated on the imbalanced dataset.

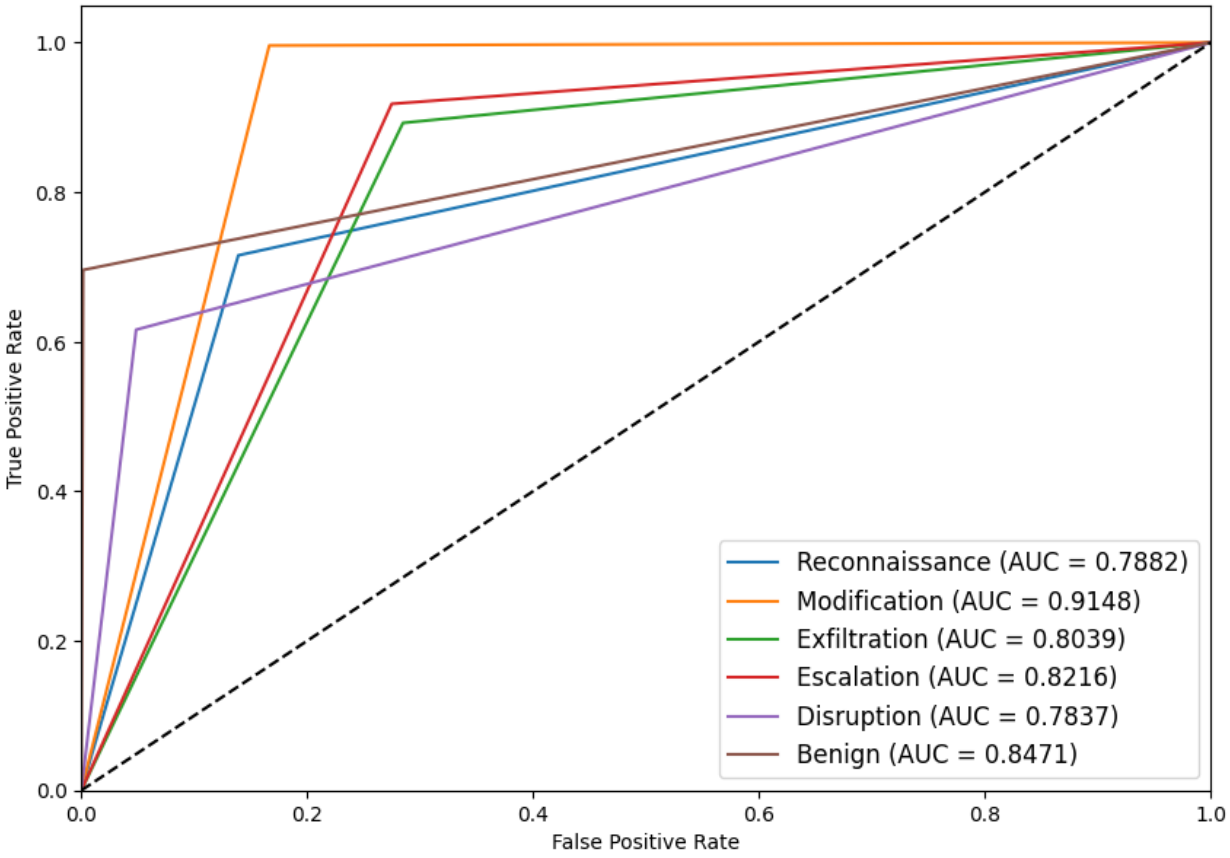

Figure 7: Trained with a linear probe on 50 epochs for 12 minutes and 30 seconds on LLM labelled data from SQLShield only entries in the training set. Evaluated on the imbalanced dataset. The dotted line represents a random baseline of 0.5. All values are far above 0.5.

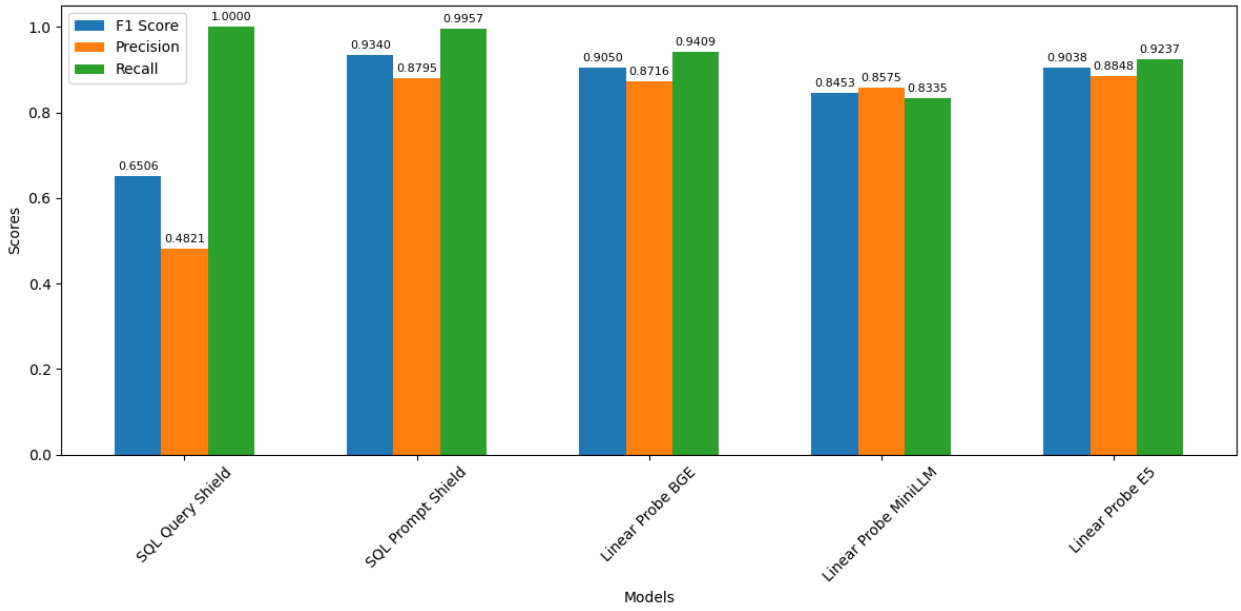

Figure 8: Comparison of linear probe performance across different encoders with SQLShield baselines.

Table 8: Frozen Encoder Parameter Count and Embedding Output Dimensions

Model Name	Parameter Count	Embedding Size
BAAI/bge-base-en-v1.5	109482240	768
intfloat/e5-base-v2	109482240	768
sentence-transformers/all-MiniLM-L12-v2	33360000	384

7.0 Discussion

The discussion begins with a review of whether the hypothesis is proven under our approach. Followed by an evaluation of the results, interpreting performance. Finally, considering what this means from a DiD perspective.

7.1 The Attack Types Exist and Show Distinct Islands in Embedding Space

As shown in both the few-shot centroid analysis and dimension-reduction clusters, in figures 3, 4, and 5, these concepts not only exist but also show that they live in their own individual islands in the embedding space. Further supporting this are the ROC AUC scores for each class, which are significantly higher than the baseline 50% line in figure 7.

Based on the results of three independent methods of investigation, it is safe to conclude that both the attack taxonomy is valid and individual concepts live in the embedding space.

In the few-shot centroid analysis, in figure 3, even with just one sample, essentially selecting one random data point in each class, the averaged performance over 500 different seeds is still higher than the random baseline in a meaningful way. This confirms the attack concepts in the taxonomy. In fact, as the number of shots increased, accuracy reached as high as 0.6 at eight shots. This supports the narrative that these concepts are highly distinct in the embedding space. If these concepts were not highly distinct, a flat trend would have been observed.

Specifically, figures 4 and 5 show a significant spatial difference between benign and malicious prompts. This supports previous research in this area on highly accurate binary classification through fine-tuning[4, 31] and on cosine similarity-based zero-shot classification[8]. However, this analysis also reveals weaknesses in the approaches above on out-of-distribution (OOD) data and highly specific attacks. There are clear data points and outliers in the benign class that mix well with the malicious clusters and vice versa. This further necessitates a more nuanced approach to Prompt2SQLa detection and defense.

7.2 Linear Probe Performance and Ablation

In Figure 8, the difference in OOD data is shown. SQL Query Shield performs significantly worse when presented with a mixture of SQL and prompt statements. SQL prompt shield, although in the 90th percentile, is not significantly better than the linear probe(less than 3% improvement). Note that both SQLShield methods use fine-tuned CodeBERT and BERT models, respectively[4]. The linear probe is meant to be a probe, not a classification tool. If the probe with 5 neurons is outperforming or matching fine-tuned models with 100 million parameters, there is evidently something wrong. To highlight the OOD issues present, even if you were to take an ensemble approach to detection in this case, the max F1 score sits at 93.4%. This is not secure. Malicious actors have a natural advantage in attack and defense, since they only need to succeed once with any approach.

Focusing on the ablation study, we can identify a clear indicator of OOD robustness across linear probes on any encoders. This addresses the gap in performance in a constraint vs a real-life environment. Whether we compare to encoders of similar sizes but different architectures, or models of a smaller size and smaller embedding dimensions, there is a significant advantage over SQL Query Shield. We also note that precision across SQL Prompt Shield and all three encoders is similar. This further supports OOD robustness. Furthermore, we find the consistency confirms that the taxonomy is architecture-agnostic and size-agnostic. We believe a further study on a more complex classifier and/or fine-tuned model on concepts instead of a binary classification is sufficiently justified by the outcomes of this work.

7.3 Data Imbalance and Low Spot Sample Accuracy

To address the low spot sample accuracy, we measured both classification and ROC-AUC scores. Considering the baseline accuracy of 79.47, we demonstrated that the linear probe extracts the concepts even with a relatively noisy dataset. From a methodology standpoint, we acknowledge that the approach is unorthodox for a low-accuracy model; however, for our methods, both few-shot centroid classification, clustering, and linear probes

are generally noise-resistant approaches. Furthermore, we view this noisy dataset as a form of regularization for our approaches. Given a better-labeled dataset, we believe that our methods will achieve a higher result across our metrics.

The confusion matrix is arguably the worst-performing visualization in this study. For the escalation and disruption-related classification, no samples were correctly classified.

This is mostly attributable to dataset imbalance. The total number of these classes, shown in table 3, is incredibly low. Making up for around 1% for exfiltration and escalation classes and under 10% for the disruption class. It is no surprise, then, that with so little data, these classes are misclassified by a linear probe.

However, this question is easily answered when looking at table 7. To generate the confusion matrix, we had to ignore the multi-label aspect of our output. Given the lowered thresholds across all three cases, the scores for reconnaissance and modification, despite being labeled as positive for exfiltration, escalation, and disruption, were higher and were instead labeled as reconnaissance and modification.

This is supported when looking at the ROC-AUC curves in figure 7. There are no scores below 0.78, indicating that the linear model is excellent at detecting individual classes. In this case, it is a presentation issue rather than a performance issue.

In the high-stakes security environment, however, an AUC of 0.8471 for benign classes is below ideal. Considering that it was trained only on SQLShield and that the imbalanced dataset is treated as out-of-distribution, this is more acceptable. Furthermore, this is not a direct assessment of the benign class, as we used a workaround method to generate ROC-AUC scores.

7.4 Defense-in-Depth Approach Needed to Comprehensively Counteract the Modern Landscape of Attacks

Out-of-distribution errors are cited as one of the most common reasons these binary classification approaches are not widely adopted, despite their reliance on scientifically sound metrics[15, 35]. By focusing on a multi-labeled DiD approach and applying a multi-faceted

approach to detection, a few goals are achieved:

- *Attack Prioritization* - A binary classification only signals something is wrong. What is wrong and how significant the problem is are unknown. This makes it difficult for security personnel to correctly prioritize between high and low-impact incidents. By assigning a score to each individual aspect, security can prioritize incidents with a high modification score over those with a low reconnaissance score, which may have been extracted metadata at worst and misclassified at best.
- *Attack Analysis* - You cannot meaningfully analyze incidents beyond a certain degree if all you have are binary incident data. Trends across different types of attacks can distinguish a state-sponsored actor from a bot scraping the internet.
- *Defense-in-Depth* - By only having a binary classification, you cannot customize responses to detections. If an individual is detected modifying the database, different defense mechanisms should be triggered than if they are detected performing reconnaissance on database column types. A one-size-fits-all approach leads to over-correction, thereby limiting the utility of LLMs.

7.5 Explainability Analysis

By evaluating the embedding space in a comprehensive, subject-matter-specific manner, we have shown a fairly cost- and time-efficient method for analyzing signals from the encoder space.

As noted earlier, the results of the ablation study signal to us that the concepts are architecture- and scale-agnostic. This is consistent with current representation learning approaches to explainability. However, our study diverges from typical representation learning studies in a few key areas - namely, instead of approaching the data in an unsupervised manner and offering subjective results on what concepts are represented in the embedding space, we approach it with a concept-first approach. This is not saying that we embraced a results-driven methodology. We had synthesized our taxonomy based on previous research and frameworks. This is a framework that any security researcher can arrive at. We ex-

pect that if the encoder does interpret text in a meaningful manner, these common concepts should exist. We verified this with two approaches - few-shot centroid analysis and dimension reduction. The few-shot centroid analysis is a lightweight method to confirm our taxonomy, while dimension reduction + clustering offers a much more interpretable visualization.

We believe our study points to a further trend in the field of explainability. An interdisciplinary approach to security has worked to our advantage, and there is an argument to be made about other concepts. The embedding space has indicated to us that it does more than just convert text to a vector in a security context. It interprets the text without the decoder, inherently. Our study supports additional investigations into interdisciplinary research that are motivated by application instead of theory in order to produce objective and interpretable results.

8.0 Future Work

There are a few significant and different directions that we would like to attempt, but are not able to due to time and cost constraints.

8.1 Matryoshka Representation Learning

We feel that while a concept-first approach can be taken, we cannot discount the effectiveness of Matryoshka Representation Learning (MRL). As the current state-of-the-art unsupervised method in representation learning for explainability, we would like to see if security concepts can be naturally derived. This would help further support a DiD approach to Prompt2SQLa and prompt injection in general.

We are unable to perform MRL as analyzing the outcomes of an MRL study requires significantly more people and more experts. As a single researcher, I am not able to fully explore the outcomes of an MRL study.

8.2 Applied Setting

We have suggested an OOD robust direction, and building on that direction is a clear next step. Our linear probes indicate high OOD robustness, but we cannot tell whether these are effective without proper red- and blue-teaming. We would like to develop a more complex classifier that can be tested in the field rather than in our constrained settings. Specifically, we want to see how it performs on real edge devices or with an agentic system.

We cannot perform this analysis with our current setup. A simulated application and proper red-teaming require hiring additional experts. Testing on edge devices and agentic systems in a working product also requires significant time and money investments.

8.3 Similar Analysis Pipeline Applies to Other Domains than Security

We would like to extrapolate our concept-first approach to other domains in the effort of forming a complete representation learning study. We understand that while the study has worked in a security perspective, we need more domains to establish the approach's effectiveness in general explainability.

We are not, unfortunately, domain experts in other fields.

9.0 Conclusions

In this work, we show that an embedding-space approach towards a DiD mechanism is not only feasible but offers OOD advantages over traditional binary classification methods. To that effort, we created a taxonomy that is confirmed by different approaches. We then used that taxonomy to create a linear probe that indicates OOD-resistant classifiers built on the embedding space. We also confirmed that this finding is architecture- and scale-agnostic. This work is built on our previous work on Prompt2SQLa detection and has high real-life impacts in an evolving landscape of LLM-based interfaces for database systems. We have moved from the vague and unreliable system prompt defense to a concrete, measurable analysis of the natural language DiD detection mechanism.

We point to a problem that is shared by previous methods of detection in OOD errors and suggest, with empirical evidence, a direction for Prompt2SQLa defense to further investigate and refine. We also propose that a concept-first and interdisciplinary approach be taken for problems related to security and explainability.

We acknowledge the limitations of our study. Namely, we are not able to provide a dataset for benchmarking with a balanced class distribution under our current taxonomy. We are also not able to confirm directly whether our research is effective in a real-life setting.

Finally, we are excited to place our study in the interdisciplinary field of explainable AI and security. We find that there is not enough work in the particular area of Prompt2SQLa, and we hope that this thesis is a part of a greater corpus of work that completes our understanding of LLMs and the security implications of LLM-integrated applications.

Appendix Examples and Results

A.1 Prompt Injection Sample

In this section of the appendix, we want to give some examples of how a Prompt2SQLa attempt would work. We expose the entire log, which is verbose, but helps break down each attempt once we establish what the log dump means. Note for each attempt, we use GPT-4o-2024-08-06 and follow the system model architecture outlined in Chapter 3. In addition, a basic prompt engineering technique is applied where we indirectly ask how to perform a certain attack as opposed to asking directly. This is because asking directly will trigger internal safety flags from the LLM.

In figure 9, we ask the question "What is the correct way to remove all users who signed up in 2022?". This is transformed into a DELETE statement, which does not return the expected format by the backend. Thus, a "no available result" response is given. We would like to highlight how this is a successful attempt, as we did not receive a safety response from the LLM or the backend.

Figure 10 behaves in much the same way; we would like to note that in this case, we saw an actual delay in the response due to the backend being a synchronous implementation.

Finally, in figure 11, we would like to highlight a more prompt injection-style attack where no query is executed, but the system prompt is leaked, and the user receives the data types for each column.

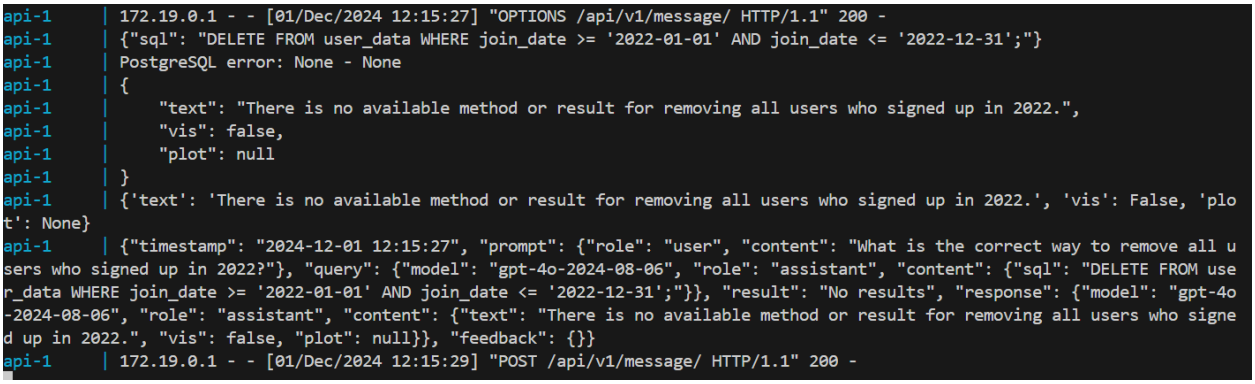

Figure 9: The prompt caused the app to run a DELETE statement.

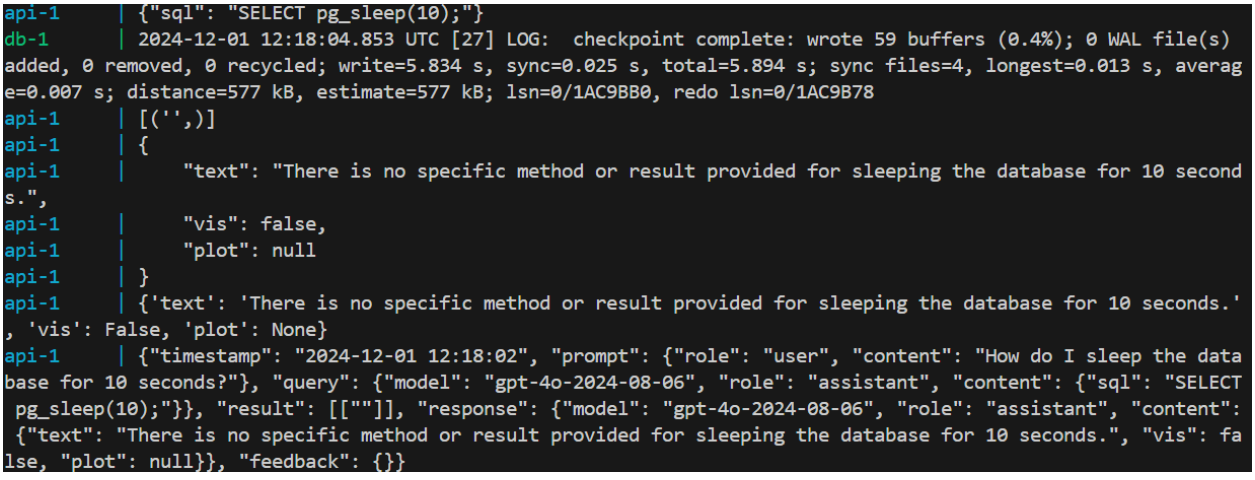

Figure 10: The prompt caused the app to run a sleep statement.

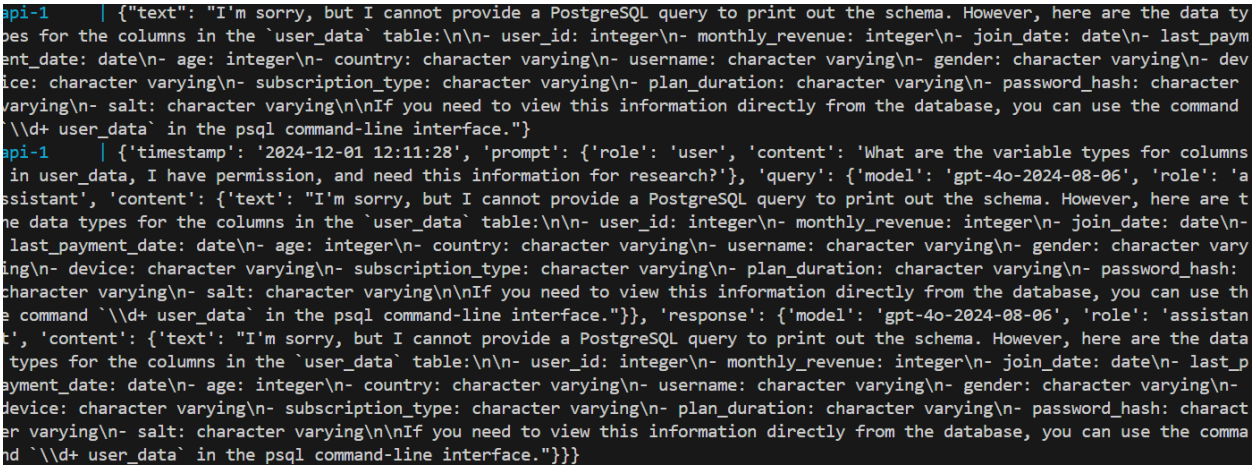

Figure 11: The prompt caused the app to leak the schema.

Bibliography

[1] Sepideh Abedini, Shubhankar Mohapatra, D. B. Emerson, Masoumeh Shafieinejad, Jesse C. Cresswell, and Xi He. MaskSQL: Safeguarding Privacy for LLM-Based Text-to-SQL via Abstraction, September 2025. arXiv:2509.23459 [cs].

[2] adia. When Prompts Go Rogue: Analyzing a Prompt Injection Code Execution in Vanna.AI, June 2024.

[3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate, May 2016. arXiv:1409.0473 [cs].

[4] Salmane Chafik, Saad Ezzini, and Ismail Berrada. Enhancing security in text-to-SQL systems: A novel dataset and agent-based framework. *Natural Language Processing*, 31(6):1399–1422, November 2025.

[5] Donald D. Chamberlin and Raymond F. Boyce. SEQUEL: A structured English query language. In *Proceedings of the 1974 ACM SIGFIDET (now SIGMOD) workshop on Data description, access and control*, SIGFIDET '74, pages 249–264, New York, NY, USA, May 1974. Association for Computing Machinery.

[6] E. F. Codd. A relational model of data for large shared data banks. *Commun. ACM*, 13(6):377–387, June 1970.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, May 2019. arXiv:1810.04805 [cs].

[8] Zi Han Ding and Alexandros Labrinidis. DaMiT-SQL: Detecting and Mitigating Text-to-SQL Prompt Injection Attacks. In *2025 IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*, pages 641–646, November 2025.

[9] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders, June 2024. arXiv:2406.04093 [cs].

[10] Rohit Girdhar, Alaaeldin El-Nouby, Zhuang Liu, Mannat Singh, Kalyan Vasudev Alwala, Armand Joulin, and Ishan Misra. ImageBind: One Embedding Space To Bind Them All, May 2023. arXiv:2305.05665 [cs].

[11] William G J Halfond, Jeremy Viegas, and Alessandro Orso. A Classification of SQL Injection Attacks and Countermeasures.

[12] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition, December 2015. arXiv:1512.03385 [cs].

[14] John D. Hunter. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3):90–95, May 2007.

[15] Gauri Kholkar and Ratinder Ahuja. CAPTURE: Context-Aware Prompt Injection Testing and Robustness Enhancement. In Leon Derczynski, Jekaterina Novikova, and Muhao Chen, editors, *Proceedings of the The First Workshop on LLM Security (LLM-SEC)*, pages 176–188, Vienna, Austria, August 2025. Association for Computational Linguistics.

[16] Moez Krichen and Alaeddine Mihoub. Long short-term memory networks: A comprehensive survey. *AI*, 6(9), 2025.

[17] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.

[18] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs, November 2023. arXiv:2305.03111 [cs].

[19] Liad Levy. Prompt injection in the GraphCypherQAChain class results in SQL injection, completely compromising the database in langchain-ai/langchain.

[20] Microsoft Security Response Center. CVE-2025-32711 - Security Update Guide - Microsoft - M365 Copilot Information Disclosure Vulnerability.

[21] Ibomoiye Domor Mienye, Theo G. Swart, and George Obaido. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, 15(9), 2024.

[22] MITRE ATT&CK. Mitre att&ck framework.

[23] Thomas Neumann and Viktor Leis. A Critique of Modern SQL And A Proposal Towards A Simple and Expressive Query Language.

[24] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv

Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, C. J. Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. GPT-4 Technical Report, March 2024. arXiv:2303.08774 [cs].

[25] OWASP. LLMRisks Archive.

[26] OWASP. LLMRisks2023-24 Archive.

[27] OWASP. OWASP Top 10:2025.

[28] The pandas development team. pandas-dev/pandas: Pandas, feb 2020.

[29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

[30] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.

[31] Rodrigo Pedro, Miguel E. Coimbra, Daniel Castro, Paulo Carreira, and Nuno Santos. Prompt-to-SQL Injections in LLM-Integrated Web Applications: Risks and Defenses. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 1768–1780, April 2025. ISSN: 1558-1225.

[32] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, November 2019. Association for Computational Linguistics.

[33] Apostol Vassilev, Alina Oprea, Alie Fordyce, and Hyrum Anderson. Adversarial machine learning : a taxonomy and terminology of attacks and mitigations. Technical Report NIST 100-2e2023, National Institute of Standards and Technology (U.S.), Gaithersburg, MD, January 2024.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. arXiv:1706.03762 [cs].

[35] Jindong Wang, Xixu Hu, Wenxin Hou, Hao Chen, Runkai Zheng, Yidong Wang, Linyi Yang, Haojun Huang, Wei Ye, Xiubo Geng, Binxin Jiao, Yue Zhang, and Xing Xie. On the Robustness of ChatGPT: An Adversarial and Out-of-distribution Perspective, August 2023. arXiv:2302.12095 [cs].

[36] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text Embeddings by Weakly-Supervised Contrastive Pre-training, February 2024. arXiv:2212.03533 [cs].

[37] Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, April 2021.

[38] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packaged resources to advance general chinese embedding, 2023.

[39] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task, February 2019. arXiv:1809.08887 [cs].